

Modulo 2 Look-and-say sequence

This is a document analysing a variation of the Look-and-say sequence. In essence, the sequence works by sorting runs of identical digits of a number to modulo 2 followed by that digit. This means with an initial term like 1, the sequence goes as follows:

The first term 1 has one 1, so it is read off as "1 (mod 2) followed by 1" or 11.
The next term 11 has two 1's, so it is read off "2 (mod 2) followed by 1" or 01
The next term 01 has one 0, and one 1, so it is read off as "1 (mod 2) followed by 0, 1 (mod 2) followed by 1) or 1011.

The next terms thus are:

111001

110011

010001

And so on...

This 'initial term' being 1, will grow infinitely in length.

Assuming X to be used to represent any digit exclusively 2-9 in a number, we can say that any combination of exclusively these specific digits in an initial term:

0,

X,

10, (where there is an odd amount of 1's, followed by an odd amount of 0's)

1X (where there is an odd amount of 1's, followed by an odd amount of X's)

will form degenerate sequences where eventually one term will equal the last.

As an example:

0X101X

101X1110111X

1110111X1110111X

1110111X1110111X

Any combination of such mentioned digits with additionally 1 or just 1 by itself for the initial term will form a infinitely growing sequence. For example the initial term 010110 is made up of 0, 10, 1 and 10. Since there is at least one 1, this initial term will grow infinitely.

010110

1011100110

111011000110

111001100110
110001000110
011011100110
10011011000110
...

Splitting behavior of the digit X

The digit X has the unique property in a term such that it separates digits on its left interacting with digits on its right, which acts quite similar to the Look-and-say sequence where a term split off into specific elements that never interact with each other again. Taking an example like 11X1, the X effectively splits the growth of this term into 11X and 1, which grow independently and never interact with each other. This is due to the fact that the rightmost digit of X can only ever be 1 or 0, thus never interacting with it again.

11X1
011X11
10011X01
1100011X1011

The digit X otherwise acts the same as the digit 0, as they both generate degenerate sequences as mentioned earlier when in isolation. Also, if X is the last digit of a specific term, it can be replaced with a 0 with no difference to the growth of the sequence otherwise. Finally, the digits X can only appear if the initial term has X in it, this is as the (mod 2) function only returns 1 or 0.

Algorithm to produce the sequence

This code in Python can be used to generate an array (terms) of all the terms of the sequence starting from the initial term (currentterm). How many times the for loop runs decides how many terms there are in the array excluding the first term.

```
currentterm = "1"
count = 0
terms = [currentterm]
for y in range(4000):
    newterm = ""
    digit = currentterm[0]
    for x in range(len(currentterm)):
        if digit == currentterm[x:x+1]:
            count += 1
        else:
            newterm += str(count%2) + digit
            count = 1
            digit = currentterm[x:x+1]
    newterm += str(count%2) + digit
    count = 0
    currentterm = newterm
    terms.append(currentterm)
print(terms)
```

Recurrence Conjecture

The recurrence Conjecture states that for every single term after the first (no matter what initial term is used so long as there is no X digit), that term will always appear again at the end of another arbitrary term at set intervals.

For example, looking at the initial term 110:

110

0110 ← looking at this term

100110 ← it will appear at the end of another term

11000110

01100110

1001000110 ← looking at this other term

110011100110

010011000110

10110001100110

1110011001000110 ← it will also appear at the end of another term

...

Giving a larger example, here is one arbitrary term from the initial term 011:

0100111001001110111001001110010001001110010001001110010001001110
010001001110111001

You will find it at the end of these terms:

0100111001000100111001000100111001000100111001000100111001001110
1110010011100100010011100100010011100100010011100100010011101110
01

,

010011100100010011100100010011100100010011100100010011100100010
0111001000100111001000100111001000100111001000100111001000100111
0010001001110010001001110010011101110010011100100010011100100010
01110010001001110010001001110111001

and:

01001110010001001110010001000100010001001110010001001110010
00100111001000100111001000100010001000100111001000100111001
000100111001000100111001000100111001000100111001000100111001000
100111001000100111001000100111001000100111001000100111001000100
111001000100111001000100111001001110010011100100010011100100
010011100100010011100100010011101110010011100100010011100100
01001110010001001110010001001110111001

Every term N after the first term, according to the conjecture, reappears at the end of another term at set constants, the constant being K-N where term K is the first term after term N with term N in the end. Thus a equation can be written as such:

$$T_{N+AB} = (\text{any possible arrangement of 1s and 0s}) || T_N$$

Where:

- T_N is our arbitrary term N where N is greater than 1
- A is a positive integer
- B is a constant K - N for which the term T_K is the first term after T_N that has T_N on its end in the sequence.

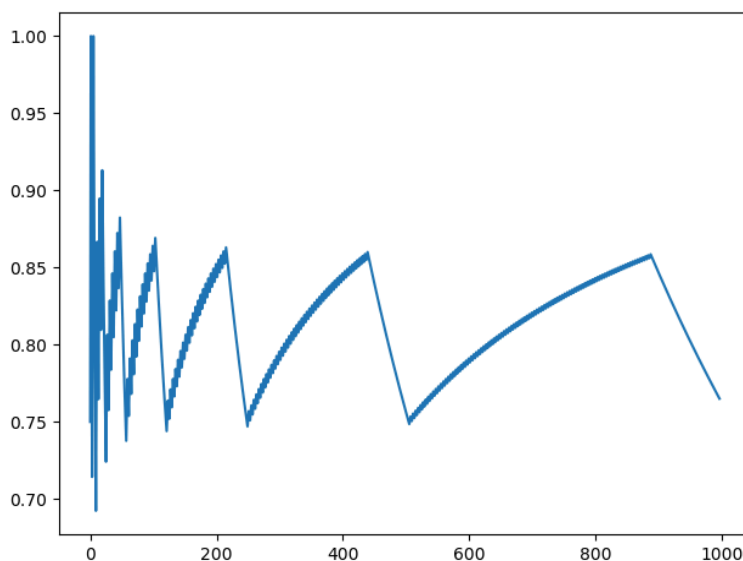
The reason the initial or first term is excluded from this theorem is because of two possible features of the first term that can make it impossible to generate in later terms. This is either because the first term might contain a run of identical digits greater than 3 like 11111... which is not possible to occur after the first term; or because it could be made up of an redundant statement like 0101 which its theoretical previous term would read out "2n (mod 2) = 0 followed by 1, 2n (mod 2) =0 followed by 1) to create. The problem is, since both statements are referencing the same digit (1), the sorting rule would simplify this to a single "2n (mod 2) = 0 followed by a 1", thus making 0101 impossible to appear in any later term. This does show in general that after the first term, every second digit in any term must be exclusively different (which is the same property the original Look-and-say sequence also holds).

Asymptotic Linear Growth Rates of lengths of initial terms

In the Look-and-say sequence, the length of each successive term eventually grows around 30% greater than the last no matter what initial value you start with. In this sequence, since the length of a term can only grow by +0, +1 or +2 from the last (assuming there is no digit X in the initial term), the ratio between two consecutive terms eventually reaches 1 asymptotically. However, depending on the initial term, the linear growth rate in terms of length can be quite different as time goes on. For the linear growth rate over time of an initial term, it ranges from 0 to less than 2 (assuming there is no digit X in the initial term). If the digit X is included in the initial term, and splitting behaviour is created as shown previously in a term like 1121, the linear growth rate can be infinitely high.

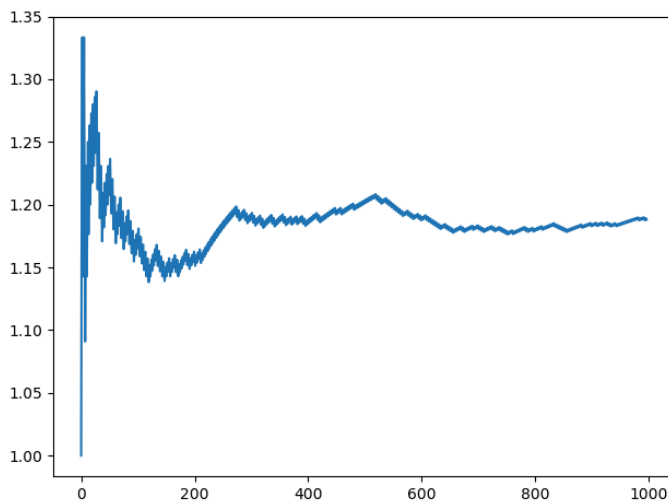
Here are some graphs that showcase this:

Graph of linear growth rate for initial term 1 over time:



The linear growth rate eventually reaches approximately 0.7952204779522047.

Graph of linear growth rate for initial term 110 over time:



The linear growth rate over time here eventually reaches approximately 1.1965803419658034, which is much greater than that for initial term 1. Which means that on average, each term starting from 110 grows a lot faster than that for 1. Also, the linear growth rate here over time also converges a lot quicker than that for initial term 1. This is probably due to the fact that for the initial term 110, the length of each term increases much more consistently in a chaotic order while for 1, terms will constantly stop and regrow in the number bits at a growing rate over time (as will be clear in the cellular automata section). This means the value for initial term 1 takes a lot longer to converge.

Additional interest for the initial term 1

When you look at how many times the initial term 1 stays the same length in bits with each consecutive term, you get these growing values:

2, 5, 7, 11, 19, 35, 67, 131, 259, 515, 1027, ...

Which after 2 follows this iterative formula exactly:

$$I_{n+1} = 2I_n - 3$$

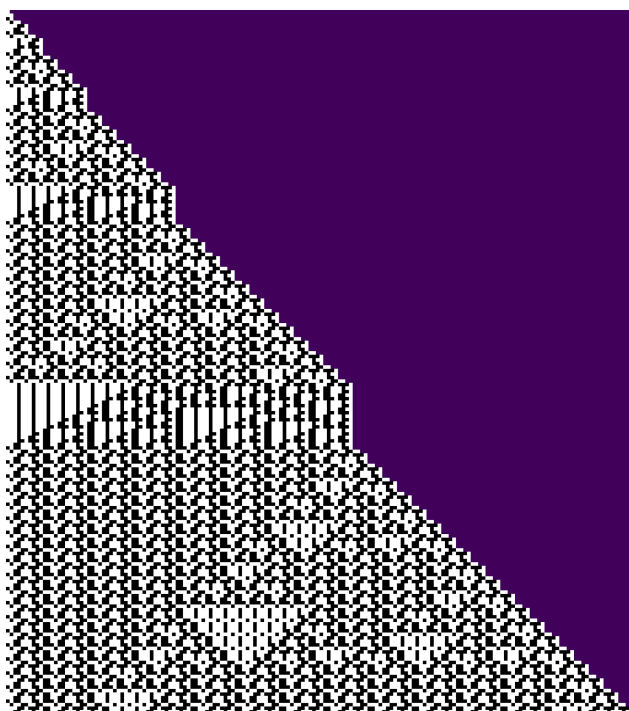
Relation to Cellular Automata and the Sierpinski's triangle

The look and sort sequence bears striking similarity to Cellular Automata systems like rule 90 as it can produce both chaotic and orderly visual patterns that closely resemble Sierpinski's triangle, this is in part due to the recurrence theorem which allows for later terms to reference smaller earlier terms. To make these visual patterns, each term of the sequence starting from the initial term is displayed under the last where 1 and 0 are represented as white and black pixels respectively for visual easiness.

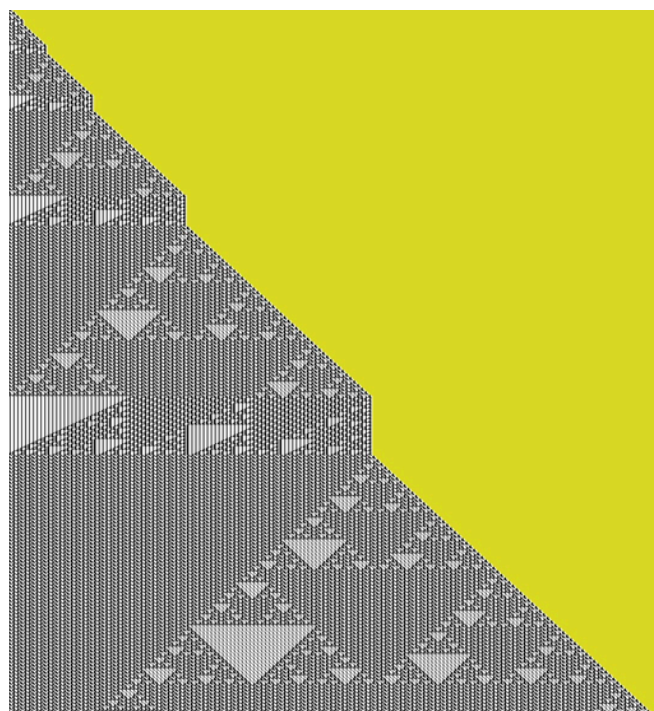
As an example starting from initial value 1:

1					
1	1				
0	1				
1	0	1	1		
1	1	1	0	0	1
1	1	0	0	1	1

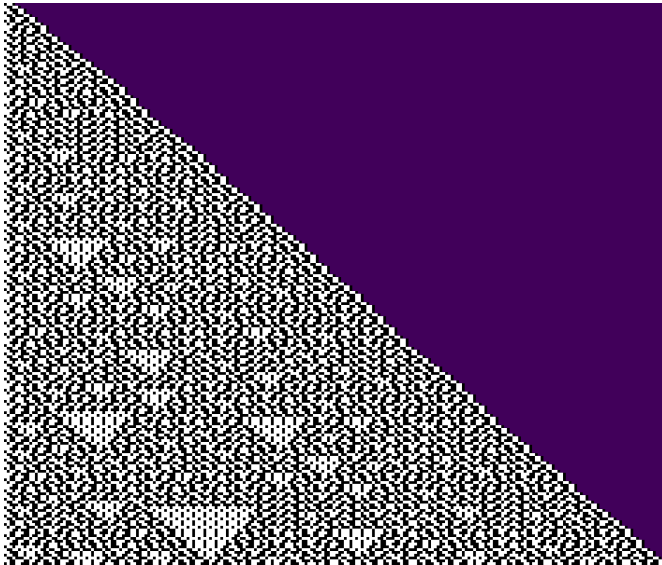
Initial term = 1, 200 iterations



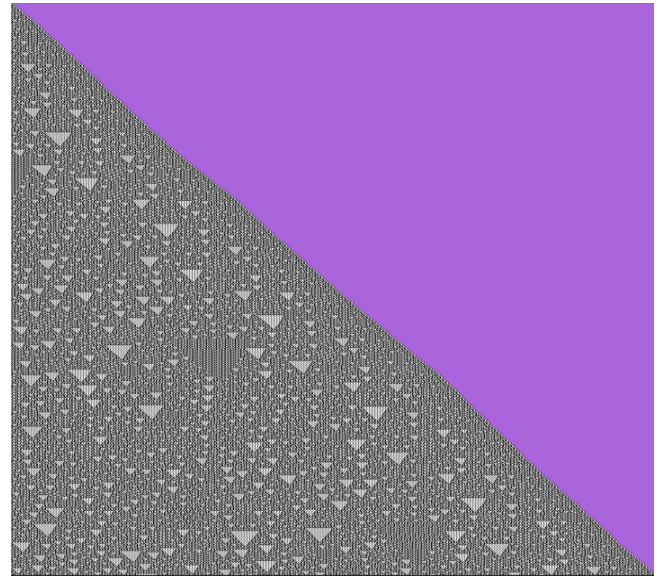
Initial term = 1, 800 iterations



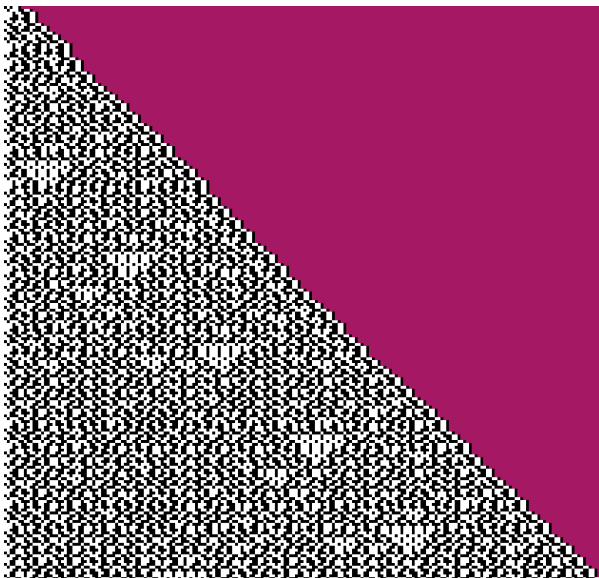
Initial term = 110, 200 iterations



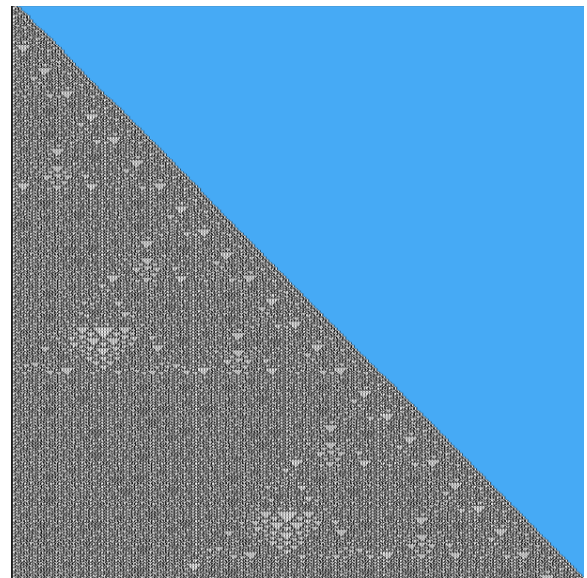
Initial term = 110, 800 iterations



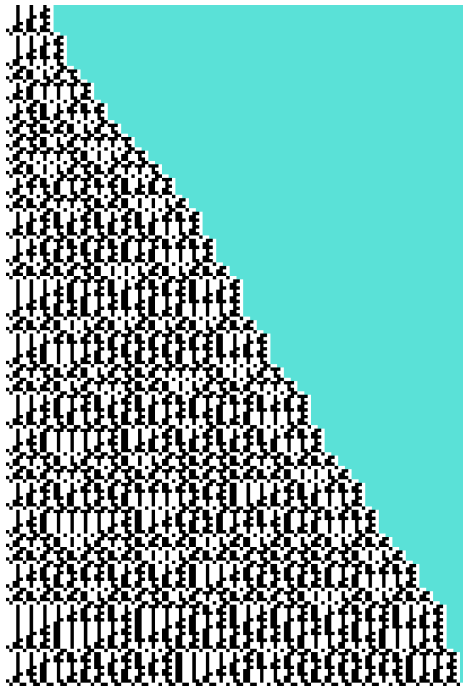
Initial term = 010110, 200 iterations



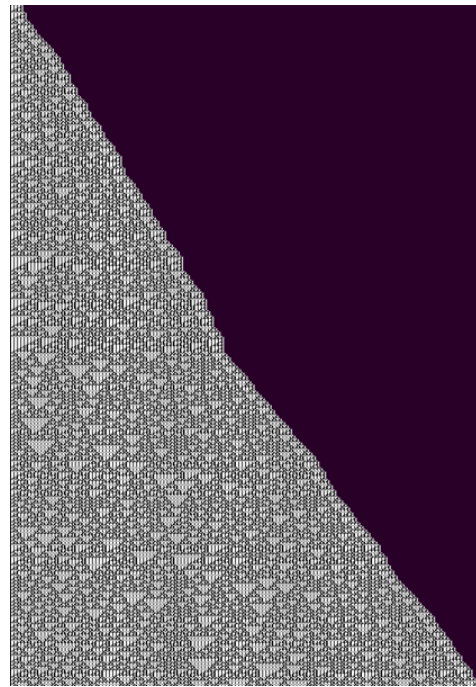
Initial term = 010110, 800 iterations



Initial term = 11101110111011, 200 iterations



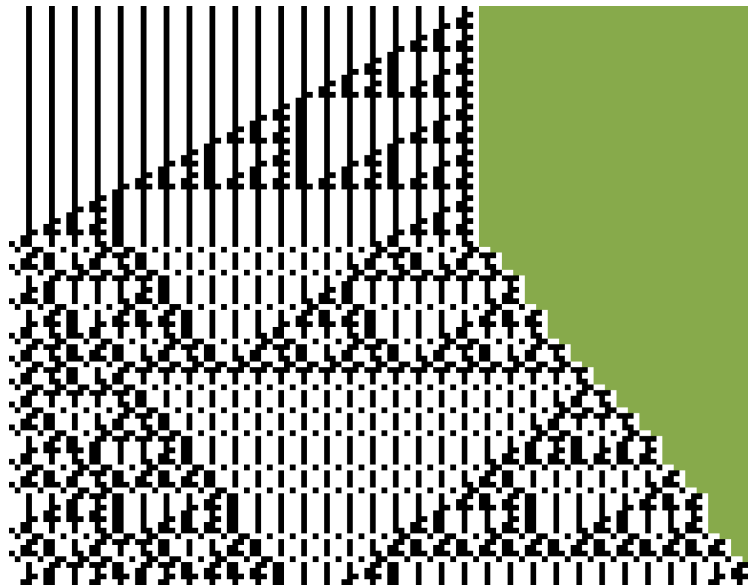
Initial term = 11101110111011, 800 iterations



As seen in these examples, the different linear growth rates of each initial term are quite apparent when looking at them visually. Some patterns form an almost perfect right-angled triangle while others either slope more steep or constantly pause and regrow. There is also the fact that some initial terms seem to create constantly reappearing and larger patterns, for example with 010110 or 1 while others don't visually create any noticeable larger patterns to point out (which mimic very similarly something like random seeds for the Rule 90 cellular automata that can often create psuedo-random growth). So far I am not sure exactly why certain seeds like 1 or 010110 do this in the first place but It will be looked at in the future.

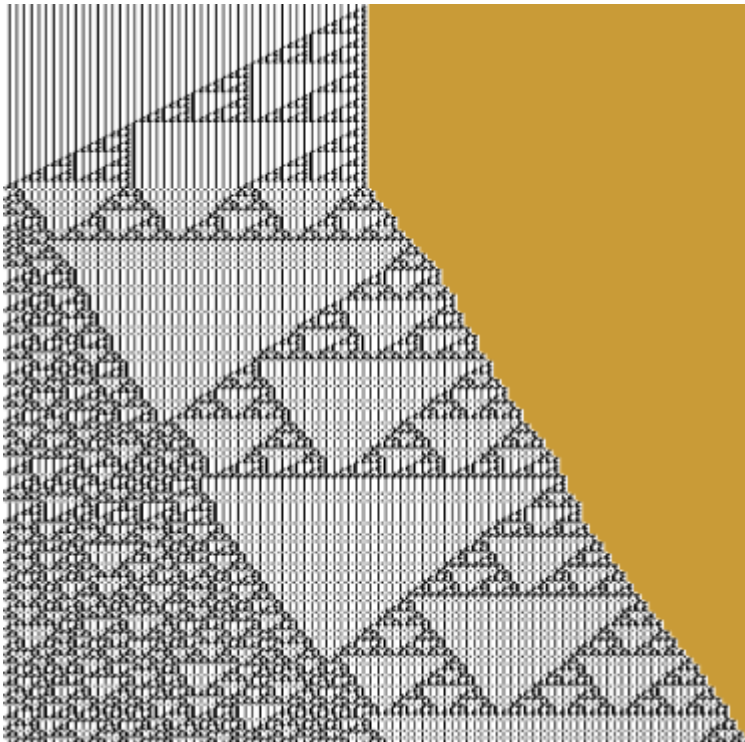
If we were to take an initial seed that only consists of 1110 with an additional 11 at the very end, we can actually form a perfect Sierpinski's triangle as the seed gets longer.

For example we can repeat 1110 20 times with an additional 11 at the end and we will get this pattern for 100 iterations:



(Note that this perfect Sierpinski's triangle will only form so long as each term is the same length in bits as the last, once the sequence starts growing in bits, the pattern will follow into pseudo-random noise.)

Here is a larger example ran over 200 iterations where you can see a more well defined one at the top:



Finally using the digit X, you can create 2 separate patterns that grow on top of each other leading to some very strange looking patterns.

For example here is the initial term 1121 over 400 iterations:

